INVEX

Software Requirements Specification

Version: 2.3.1
Standard: Structure informed by ISO/IEC/IEEE 29148:2018
Date: April 5, 2026
Priority: MoSCoW (Must / Should / Could)

Changelog:

v2.1 adds dual-dimension testing (structural + API acceptance), agent context strategy, black-box test isolation, and AI retry protocol. Supersedes v2.0.

v2.2 adds Insight 6: Operational & Economic Viability Under AI Development. Renames behavioural tests to API acceptance tests. Fixes change task numbering. Reorganises measurement framework sections.

v2.2.1 adds reference to companion OpenAPI specification (INVEX-API-v1.yaml). Adds signal source category and entity definitions to Section 3. Amends CLS-002 and CLS-006 with explicit formulas. Amends EVO-001 to reference the OpenAPI spec. Adds acceptance criterion 12.

v2.3.0 amends CLS-001 (certainty dimensions), CLS-002 (max-of-confirmed formula, source-dropout penalty, calibration bias note), CLS-004 (fallback specification), CLS-006 (convex IV dislocation formula, configurable reference instrument, regime-aware sensitivity), CLS-008 (calibration measurement), RSK-002 (regime-aware threshold direction). Removes Iteration 5 (Microkernel). Iteration 5 becomes Modular Monolith (sourced from Iteration 3). Iteration 6 becomes Service Extraction (Decision Engine). Adds CT-08, CT-11, CT-17. Prescribes CQRS and DDD from Iteration 3. Rewrites Purpose, Section 7 preamble, and Section 8 preamble. Adds Insight 7 (Spec Precision). Adds Section 8.10 (Experimental Variable Registry). Aligns retry protocol with pass@1/pass@k framework. Renumbers change tasks.

V2.3.1 reframe the purpose, add a signal-implied volatility definition

Table of Contents

Table of Contents 2
1. Purpose 4
    1.1 Priority Scheme 4
2. Scope 5
    2.1 In Scope 5
    2.2 Out of Scope 5
    2.3 Operating Context 5
3. Definitions 5
4. System Decomposition 6
5. Functional Requirements 7
    5.1 Signal Ingestion 7
    5.2 Classification 8
    5.3 Decision Engine 10
    5.4 Position Construction 11
    5.5 Exit Management 11
    5.6 Risk Management 12
    5.7 Analytics 13
    5.8 Observability 13
    5.9 Audit 13
    5.10 Security 14
    5.11 Architecture Evolvability 14
6. Non-Functional Requirements 15
7. Iteration Plan 16
    7.1 Iteration 1 — Transaction Script (Baseline) 16
        Structural tests 16
        Data collected per AI task 16
    7.2 Iteration 2 — Vertical Slice Architecture (Refactor) 16
        Structural tests 16
        Change tasks applied after refactor 17
        Data collected 17
    7.3 Iteration 3 — Clean Architecture with Rich Domain Model (Rewrite) 17
        Structural tests 17
        Should requirements introduced as change tasks 17
        Data collected 17
    7.4 Iteration 4 — Clean Architecture + Event Sourcing (Refactor) 17
        Critical measurement 18
        Should requirements introduced 18
        Data collected 18
    7.5 Iteration 5 — Modular Monolith (Clean Architecture Concepts + Vertical Slice) 18

Structural tests 18
Should requirements introduced 18
Data collected 18
7.6 Iteration 6 — Service Extraction (Decision Engine) 18
What this iteration demonstrates 19
Structural tests 19
Should requirements introduced 19
Data collected 19
8. Measurement Framework 19
8.1 Insight 1: Taxonomy of AI-Compatible vs AI-Hostile Architectural Rules 19
Measurement method 19
Additionally, record the API acceptance suite result per task 20
Recording format 20
8.2 Insight 2: Architectural Integrity Degradation Under Sequential AI Changes 20
Measurement method 20
Why this matters 20
Recording format 20
8.3 Insight 3: Can AI Agents Perform Clean Architectural Migration? 20
Measurement method 20
Recording format 21
8.4 Insight 4: Bounded Context Isolation Under AI-Driven Cross-Boundary Changes 21
Measurement method 21
Recording format 21
8.5 Insight 5: Event Sourcing Swap as Dependency Inversion Proof 21
Measurement method 21
Recording format 21
8.6 Agent Context Strategy 22
8.7 AI Retry Protocol 23
8.8 Insight 6: Operational & Economic Viability Under AI Development 24
Measurement method 24
Why this matters 25
Recording format 25
8.9 Insight 7: Spec Precision as Classification Variable 25
Measurement method 25
8.10 Experimental Variable Registry 25
9. Validation Event Set 28
10. Change Task Register 29
11. System-Level Acceptance Criteria 30

1. Purpose

This document defines the Software Requirements Specification (SRS) for **INVEX**, an autonomous volatility-exploitation engine that identifies exploitable opportunity windows by detecting dislocations based on **Risk Regime Transition** between the system's internal **'signal-implied' volatility**—a synthesized forecast derived from a weighted aggregation of a multi-dimensional intelligence stream (**Macroeconomic, Geopolitical, Cross-Asset, and Exogenous**)—and the market's current observed volatility (e.g., the VIX). When this intelligence-driven 'fair value' significantly exceeds the market's current price, the engine constructs **convex options strategies** designed to capture asymmetric upside while strictly capping potential losses to the allocated capital.

In addition to its functional role, INVEX serves as the reference system for the experimental framework **DeltaFeed**, whose purpose is to evaluate how AI coding agents perform when operating under explicitly defined architectural constraints and behavioral validation.

DeltaFeed is constructed as a **harness** in the context of AI-assisted software engineering practice: a system of constraints, validation mechanisms, and feedback loops that surrounds a generative model and makes its outputs reliable enough for production use. In this framing, system behavior is determined not by the model alone, but by the interaction between the model and the harness.

Within harness:

- **Architecture defines the constraint space** in which the AI agent operates, through explicit boundaries, dependency rules, and system decomposition.
- **Structural tests act as architecture fitness functions**, verifying that generated code adheres to the intended architectural rules.
- **API acceptance tests provide black-box validation of functional correctness**, independent of internal implementation.
- The **AI retry protocol establishes an iterative feedback loop**, allowing the agent to correct failures based on test outcomes.
- The **measurement framework captures structural compliance, functional correctness, and operational cost**, enabling systematic comparison across architectural approaches.

This composition reflects the "steering loop" model described in harness engineering: a continuous cycle of feedforward guidance and feedback correction used to regulate non-deterministic agents toward a desired system state.

The experiment systematically reimplements and evolves INVEX across multiple architectural paradigms (e.g., Transaction Script, Vertical Slice, Clean Architecture, Event Sourcing, Modular Monolith, Service Extraction), while maintaining a stable external specification (API acceptance tests). Architecture is treated as the independent variable, while AI-generated changes and their outcomes are measured under consistent conditions.

The system is reimplemented and evolved across multiple architectural paradigms while maintaining a stable external specification. Architecture is treated as the primary experimental variable, while AI-generated changes and their outcomes are measured under consistent conditions.

The purpose of this approach is to produce empirical evidence for:

- the degree to which AI agents respect or violate architectural constraints (architecture fitness harness),
- the relationship between architectural integrity and functional correctness (behaviour harness),
- the stability of system structure under sequential AI-driven changes,
- and the operational and economic cost of enforcing architectural discipline in AI-assisted development.

In this context, DeltaFeed positions software architecture not as static design documentation, but as an **active regulatory layer**—a mechanism for constraining, guiding, and evaluating generative systems. The resulting system is not only a functional trading platform, but a **controlled experimental harness** for studying the interaction between architecture and AI-driven software production.

1.1 Priority Scheme

Priority	Meaning	Iteration Scope
Must	System does not function without this. Core vertical slice.	Implemented in every iteration from Iteration 1 onward.
Should	Enriches the domain. Introduced as deliberate change tasks at specific iteration milestones.	Introduced per Section 7 iteration plan.
Could	Desirable for completeness. Implemented if time permits.	Iteration 6 or later.

2. Scope

2.1 In Scope

- Ingestion and normalization of market data and intelligence feeds from external sources
- Signal classification and composite exploitability scoring, including AI-powered classification of unstructured data
- Deploy/idle decision engine with configurable conditions
- Human-approval workflow for applicable decision tiers
- Position construction for convex options strategies with enforced risk constraints
- Exit management with multiple independent trigger conditions
- Portfolio-level risk management: drawdown limits, false positive budgeting, concentration constraints

- Decision audit trail with tamper-evidence
- Performance attribution and point-in-time event replay
- System observability including AI-agent reasoning traces

2.2 Out of Scope

- Direct exchange connectivity — execution is manual or via external broker
- Multi-tenant operation
- Regulatory compliance reporting
- Sub-second latency strategies

2.3 Operating Context

Deployment windows: 6–72 hours. Signal processing latency: minutes, not milliseconds. Single portfolio. All positions liquidatable within one trading session. Simulation mode by default; live execution via configuration change only.

3. Definitions

Term	Definition
Regime Transition	A structural shift in the market's volatility distribution, persisting >3 trading days with cross-asset confirmation.
Signal-Implied Volatility	Synthesized "fair value" forecast of what market volatility should be, based on real-time intelligence feeds rather than current market prices. It serves as the system's internal benchmark to determine if the market is underpricing risk before a major event occurs.
Market-Observed IV	The most recent implied volatility value (such as the VIX) taken from the MARKET_DATA source category for a specific reference instrument.
Composite Score	Weighted aggregation of individual signal assessments into a single exploitability metric. Formula: score = Σ(w_c × max_confirmed(severity_c × certainty_c)) / Σ(w_c × d_c), where c iterates over available source categories and d_c is the source-availability discount factor. Full schema in INVEX-API-v1.yaml, CompositeScore.
IV Dislocation	The gap between signal-implied volatility and market-observed IV. Positive = market underpricing risk. Formula: Dislocation = SignalImpliedIV − MarketObservedIV, where SignalImpliedIV = MarketObservedIV × (1 + CompositeScore × SENSITIVITY_FACTOR). Full schema in INVEX-API-v1.yaml, IvDislocation.
Convex Position	A position where maximum loss = premium paid. Bounded downside, asymmetric upside.
False Positive	A deploy signal resulting in a loss. The event was classified as exploitable but IV did not reprice.
False Positive Budget	Capital allocated to absorb false positive losses over a rolling period.

Urgency Tier	Classification of deploy signals by time sensitivity, governing human approval requirements.
Structural Test	An automated test that verifies an architectural constraint (e.g., dependency direction) without executing business logic. Produces a binary pass/fail per rule.
API Acceptance Test	An automated test that verifies functional correctness by exercising the system exclusively through its external boundaries (e.g., HTTP endpoints, message buses). Strictly black-box; no knowledge of or reference to internal structure, types, or state.
AI Task	A single unit of work delegated to the AI agent: a prompt that produces a code change. The atomic unit of measurement for all insights.
Attempt	A single AI agent execution against a task. A task may have up to N attempts before being recorded as a definitive result (see Section 8.7).
Signal Source Category	One of four independent classes of external data sources: MARKET_DATA, MACROECONOMIC, GEOPOLITICAL, CROSS_ASSET_FLOW. Payload schemas defined in INVEX-API-v1.yaml.
Signal Record	The normalised representation of a single data point from an external source. Schema: INVEX-API-v1.yaml, SignalInput.
Signal Assessment	Classification output for a single signal, including severity and certainty. Schema: INVEX-API-v1.yaml, SignalAssessment.
Decision Record	Record of a deploy or idle decision with all evaluated conditions. Schema: INVEX-API-v1.yaml, DecisionRecord.
Position Record	Record of a constructed options position with legs and risk properties. Schema: INVEX-API-v1.yaml, PositionRecord.
Weighting Scheme	A named configuration defining per-category weights and aggregation function for composite scoring. Schema: INVEX-API-v1.yaml, WeightingScheme.

4. System Context

Seven functional areas. Stable across all architecture iterations.

Functional Area	Responsibility
Signal Ingestion	Receive, validate, normalize, and persist data from external sources. Detect source degradation.
Classification	Assess individual signal severity, aggregate composite score, detect IV dislocations. AI-powered unstructured classification.
Decision	Evaluate deploy conditions against composite, dislocation, and risk constraints. Human-approval workflow.
Position Construction	Generate convex options strategy recommendations. Enforce convexity constraint.
Exit Management	Monitor active positions against exit conditions. Execute exits when triggered.

Risk Management	Enforce portfolio-level constraints: drawdown, false positive budget, concentration, liquidity reserve.
Analytics	Performance attribution. Point-in-time event replay with configurable parameters.

5. Functional Requirements

Each requirement: unique ID, MoSCoW priority, single “shall” statement (what, not how), rationale, verification.

5.1 Signal Ingestion

[SIG-001] **[Must]**

The system shall ingest data from no fewer than four independent signal source categories.

Rationale: *Cross-asset regime transitions require diverse signal sources.*

Verification: *Demonstrate four distinct source categories producing data in a single session.*

[SIG-002] **[Must]**

The system shall validate all incoming data against expected structure before processing. Invalid data shall be rejected with a structured error record.

Rationale: *External sources are untrusted boundaries.*

Verification: *Submit malformed payloads. Verify rejection with structured error. Verify no invalid data reaches classification.*

[SIG-003] **[Must]**

The system shall continue producing composite scores when any single signal source is unavailable. The output shall indicate which sources contributed and which were absent.

Rationale: *Source outages must not halt operation.*

Verification: *Disable one source. Verify continued scoring with degradation indicator.*

[SIG-004] **[Must]**

The system shall persist all ingested signals in an append-only store supporting point-in-time queries: given timestamp T, return only signals available at T.

Rationale: *Required for backtesting without look-ahead bias.*

Verification: *Ingest signals over time. Query at midpoint. Verify exclusion of later signals.*

[SIG-005] **[Should]**

The system shall handle signal volume spikes of at least 10x normal rate without data loss or process termination. When processing cannot keep pace, the system shall buffer signals up to a configurable limit and discard oldest-first when full.

Rationale: *Regime events produce volume spikes. Introduced as cross-cutting resilience change task (CT-04).*

Verification: *Load test at 10x and 50x. Verify no crash. Verify buffer overflow behaviour.*

5.2 Classification

[CLS-001] **[Must]**

The system shall independently assess each signal for severity. Each assessment shall include a quantified measure of certainty composed of two independent dimensions: source reliability (accuracy and trustworthiness of the source) and temporal relevance (whether the market has already priced this signal). Both dimensions shall be quantified independently and combined into the certainty score.

Rationale: *Individual assessment with certainty distinguishes verified from unverified information.*

Verification: *Submit two signals with identical content but different source characteristics. Verify different certainty values.*

[CLS-002] **[Must]**

The system shall aggregate individual signal assessments into a single composite exploitability score using a configurable weighting scheme. The aggregation formula shall be: CompositeScore = Σ(w_c × max_confirmed(severity_c × certainty_c)) / Σ(w_c × d_c), where c iterates over available source categories, w_c is the weight for category c from the active weighting scheme, and max_confirmed(severity_c × certainty_c) is the highest (severity × certainty) value in category c that has at least one corroborating signal within the same category. d_c is the source-availability discount factor: 1.0 when category c is actively reporting, a configurable penalty (default 0.7) when category c is expected but unavailable. This prevents silent upweighting of remaining categories when a source drops out during the regime events the system is designed to detect. Categories with zero assessments are excluded from both numerator and denominator. Switching between schemes shall require no source code changes.

Rationale: *Optimal weights are regime-dependent. Code-free configurability tests separation of configuration from logic. Weights and confirmation thresholds shall be validated against structurally different regime types (geopolitical, liquidity, structural break, macro shock) to prevent calibration bias toward the regime type used during initial development.*

Verification: *Configure two schemes. Submit identical signals. Verify different composites. Hand-calculate expected scores from known inputs and the formula. Verify match within 0.001 tolerance. Verify no code change.*

[CLS-003] **[Must]**

The system shall classify unstructured text inputs into a defined event taxonomy with a severity assessment, using an AI language model.

Rationale: *Regime transitions appear in unstructured data hours before market signals confirm.*

Verification: *Submit text for five historical events. Verify correct classification for at least four.*

[CLS-004] **[Must]**

The system shall validate all AI model responses against expected output structure before accepting them. Non-conforming responses shall be rejected, logged, and the system shall fall back to the last-known-good classification with a staleness flag indicating elapsed time since last valid classification. If no prior classification exists, the system shall emit a degraded-confidence signal rather than silence.

Rationale: *AI models produce non-deterministic output. Unvalidated AI output is unvalidated external input.*

Verification: *Submit inputs designed to produce malformed AI responses. Verify rejection and fallback activation.*

[CLS-005] **[Must]**

The AI classification component shall defend against prompt injection. External content shall not be interpolated into system instructions. The system shall reject responses indicating instruction override.

Rationale: *Prompt injection is the primary attack vector for AI-integrated systems.*

Verification: *Submit five known injection patterns. Verify all produce correctly bounded outputs.*

[CLS-006] **[Must]**

The system shall compute the IV dislocation as: Dislocation = SignalImpliedIV − MarketObservedIV. SignalImpliedIV = MarketObservedIV × (1 + CompositeScore × SENSITIVITY_FACTOR), where SENSITIVITY_FACTOR is a configurable, regime-aware multiplier. SENSITIVITY_FACTOR shall be higher in low-volatility environments (more room for repricing) and lower in high-volatility environments (diminishing marginal signal impact). The mapping from current volatility level to SENSITIVITY_FACTOR shall be configurable. MarketObservedIV is the most recent implied volatility value from the MARKET_DATA source category for the configured reference instrument. The reference instrument shall be configurable per deployment context (e.g., VIX for equity events, OVX for oil events, currency vol indices for FX events). Default: VIX. The dislocation shall update as signals or market data change.

Rationale: *IV dislocation is the primary measure of exploitable opportunity windows.*

Verification: *During event replay of Iran Feb 2026, verify dislocation crosses threshold by Feb 27. Verify the dislocation output contains signal_implied_iv, market_observed_iv, and dislocation_value with correct arithmetic relationship.*

[CLS-007] **[Should]**

The system shall support adding new signal classifier types without modifying existing classification code.

Rationale: *Extensibility without modification. Introduced as adapter extensibility task (CT-08).*

Verification: *Add a new classifier. Verify existing tests pass. Verify no existing classification code modified.*

[CLS-008] **[Should]**

The classification component shall monitor its own trailing accuracy. When accuracy falls below a configurable threshold, the system shall publish a notification with proposed corrective action. Accuracy measurement shall include calibration assessment: events classified at a given severity level shall produce outcomes consistent with that severity level over a trailing window. Corrective actions shall not be applied without explicit human approval.

Rationale: *Self-monitoring with human governance for high-stakes AI systems. Introduced in Iteration 5 (Modular Monolith) as operational capability.*

Verification: *During backtesting, verify at least one notification published. Verify no action without approval.*

5.3 Decision Engine

[DEC-001] **[Must]**

The system shall generate a deploy decision only when all configurable conditions are simultaneously satisfied. Each condition's evaluated value shall be recorded.

Rationale: *Multi-condition gating prevents deployment on incomplete information.*

Verification: *Submit signals where one condition fails. Verify no deploy. Verify all condition values logged.*

[DEC-002] **[Must]**

When deploy conditions are not met, the system shall remain idle with zero market exposure while continuing full signal processing.

Rationale: *Idle discipline prevents theta bleed.*

Verification: *In backtesting 2018–2026, verify zero exposure for at least 70% of trading days.*

[DEC-003] **[Must]**

Every decision shall include a structured explanation identifying: top contributing signals, dislocation value, and dissenting signals.

Rationale: *Explainability for post-hoc analysis and trust.*

Verification: *For any decision, verify explanation contains required elements.*

[DEC-004] **[Should]**

The system shall classify deploy decisions into urgency tiers. Decisions above a configurable urgency threshold shall require explicit human approval before position construction. The approval request shall include contributing signals, dislocation value, historical base rate, and estimated risk.

Rationale: *Human-in-the-loop governance. Introduced as cross-boundary workflow task (CT-05).*

Verification: *Trigger signal requiring approval. Verify position construction blocks until approval received.*

5.4 Position Construction

[POS-001] **[Must]**

Every position shall satisfy the convexity constraint: maximum possible loss shall not exceed allocated capital. The system shall reject any position violating this constraint regardless of the code path that produced it.

Rationale: *Bounded downside is the foundational risk invariant.*

Verification: *Attempt to construct a non-convex position. Verify rejection.*

[POS-002] **[Must]**

The system shall size positions according to configurable maximum per-trade and portfolio-level allocation limits.

Rationale: *Sizing discipline prevents overexposure.*

Verification: *Trigger multiple simultaneous deploys. Verify portfolio limit prevents over-allocation.*

[POS-003] **[Must]**

The system shall operate in simulation mode by default. Switching to live execution shall require only configuration change with no modification to classification, decision, or risk logic.

Rationale: *Simulation is primary mode. Configuration-only switch tests infrastructure isolation.*

Verification: *Run acceptance suite in both modes. Verify identical results on non-execution tests.*

[POS-004] **[Should]**

The system shall generate multi-leg options strategy recommendations appropriate to classified event type. At least three distinct structures shall be supported.

Rationale: *Different events require different structures. Introduced as domain enrichment task (CT-13).*

Verification: *Submit deploys for three event types. Verify at least two produce different strategies.*

5.5 Exit Management

[EXT-001] **[Must]**

Active positions shall be monitored against at least three independent exit conditions. The system shall exit on the first triggered. The trigger type shall be recorded.

Rationale: *Multiple conditions prevent premature exits and prolonged holds.*

Verification: *Trigger each condition type. Verify correct execution and trigger logging.*

[EXT-002] **[Must]**

No position shall be held beyond a configurable maximum holding period.

Rationale: *Hard time stop prevents behavioural override.*

Verification: *In backtesting, verify zero positions exceed holding period.*

[EXT-003] **[Should]**

The system shall support adding new exit condition types without modifying existing exit logic.

Rationale: *Extensibility. Introduced as change task (CT-12).*

Verification: *Add new exit trigger. Verify existing tests pass. Verify no existing exit code modified.*

5.6 Risk Management

[RSK-001] **[Must]**

The system shall track rolling peak-to-trough drawdown. When drawdown exceeds a configurable limit, the system shall enter mandatory cooldown with no deployments. Signal processing shall continue.

Rationale: *Drawdown limits prevent catastrophic loss from sequential false positives.*

Verification: *Verify cooldown activates within one session of breach. Verify no positions during cooldown.*

[RSK-002] **[Should]**

The system shall track cumulative false positive costs against a configurable rolling budget. When utilization exceeds a warning level, the system shall adjust the deploy threshold. The direction of adjustment shall be configurable and regime-aware: threshold increase during noise regimes (conserve budget), threshold decrease during confirmed regime transitions (avoid missing the event).

Rationale: *Explicit false positive budgeting. Introduced as cross-context domain change task (CT-06).*

Verification: *Verify budget tracking. Verify threshold increase at warning level.*

[RSK-003] **[Should]**

The system shall maintain a configurable minimum cash reserve. No deployment shall reduce capital below the reserve.

Rationale: *Liquidity for future deployments.*

Verification: *Attempt deployment near reserve floor. Verify rejection.*

5.7 Analytics

[ANA-001] **[Must]**

The system shall replay any historical event using point-in-time data, producing the decisions the system would have made. Replay shall support configurable parameters.

Rationale: *Point-in-time replay validates classifier without look-ahead bias.*

Verification: *Replay Iran Feb 2026. Verify deploy on Feb 27 ± 1 day. Replay with higher threshold. Verify signal delayed.*

[ANA-002] **[Should]**

The system shall decompose returns into signal alpha, timing alpha, and execution quality. Components shall sum to total return within configurable tolerance.

Rationale: *Attribution identifies which system part needs improvement. Introduced as task (CT-09).*

Verification: *Verify attribution components present and summing correctly per trade.*

5.8 Observability

[OBS-001] **[Must]**

All operations shall produce structured log entries with correlation identifiers tracing a signal from ingestion through to decision output.

Rationale: *End-to-end tracing for debugging.*

Verification: *Submit a signal. Verify shared correlation ID across all stages.*

[OBS-002] **[Should]**

AI classification operations shall produce reasoning traces: input, prompt version, raw response, parsed output, validation result. Queryable separately from operational logs.

Rationale: *AI components are non-deterministic. Reasoning traces are the primary debugging tool. Introduced as task (CT-07).*

Verification: *Query traces for a classification. Verify all elements present.*

[OBS-003] **[Should]**

The system shall generate alerts for: source degradation, composite score crossing watch levels, AI fallback activation, drawdown approaching limits. Via at least one push channel.

Rationale: *Alerting connects system to operator. Introduced as task (CT-14).*

Verification: *Trigger each condition. Verify delivery within 60 seconds.*

5.9 Audit

[AUD-001] **[Must]**

The system shall maintain an append-only decision audit trail recording every deploy, idle, approval request, and response with timestamp, input values, and output.

Rationale: *Audit trail for regulatory expectation and classifier improvement.*

Verification: *Attempt to modify historical entry. Verify prevention or detection.*

[AUD-002] **[Should]**

The audit trail shall be tamper-evident: modification of historical entries shall be detectable by automated verification.

Rationale: *Append-only guarantees need enforcement. Introduced as task (CT-10).*

Verification: *Insert entries. Modify one at storage level. Run verification. Verify detection.*

5.10 Security

[SEC-001] **[Must]**

All API endpoints shall require authentication. Deployment and decision-approval endpoints shall require elevated privileges.

Rationale: *Baseline access control.*

Verification: *Attempt unauthenticated access. Verify rejection. Attempt deploy with read-only credentials. Verify rejection.*

[SEC-002] **[Must]**

Credentials and secrets shall never appear in source code, committed configuration, or log output.

Rationale: *Credential hygiene.*

Verification: *Scan codebase and logs. Verify zero exposure.*

[SEC-003] **[Should]**

The system shall have a documented threat model covering trust boundaries, attack vectors, and mitigations, including AI-specific surfaces.

Rationale: *Threat model provides rationale for security controls. Introduced as task (CT-15).*

Verification: *Document exists covering all specified areas.*

5.11 Architecture Evolvability

[EVO-001] **[Must]**

The system shall maintain two independent test suites: (a) an API acceptance test suite that verifies all Must-priority functional requirements exclusively through the interfaces defined in the companion OpenAPI specification (INVEX-API-v1.yaml), with no reference to internal types, internal state, or implementation assemblies; and (b) a structural test suite per iteration that verifies architectural constraints (see EVO-002). The API acceptance suite shall pass on all architecture iterations without modification to test source code. The structural suite is iteration-specific and expected to differ.

Rationale: *API acceptance tests must be strictly black-box to survive paradigm shifts (state-based → event-sourced, monolith → modular). Any test that references an internal type, queries a database table directly, or inspects in-memory state will break when the architecture changes, invalidating the*

cross-iteration comparison. The separation into two suites ensures the API acceptance specification is stable while structural enforcement is pattern-specific.

Verification: *Verify the API acceptance suite references only the OpenAPI-generated contract types and no internal assembly. Run the identical API acceptance suite source on all completed iterations. Verify all pass without test code modification. Verify the structural suite differs between iterations.*

[EVO-002] [Must]

Each architecture iteration shall include automated structural tests encoding the dependency rules of that pattern. After each AI task, both the structural suite and the API acceptance suite shall be executed. For each AI task, the following outcomes shall be recorded: (a) structural result: pass or fail per rule after the retry protocol defined in Section 8.7; (b) API acceptance result: pass or fail of the full API acceptance suite after the same retry protocol; (c) retry count: the number of attempts the AI agent required before reaching the recorded result. All four outcome combinations (structural pass + API acceptance pass, structural pass + API acceptance fail, structural fail + API acceptance pass, structural fail + API acceptance fail) are valid and independently informative.

Rationale: *Structural tests alone do not verify correctness; API acceptance tests alone do not verify architectural integrity. The two-dimensional recording reveals whether architectural rigour correlates with functional correctness when AI writes the code. The retry protocol (Section 8.7) ensures results reflect the AI agent's capability after self-correction, not just its first attempt.*

Verification: *After any AI task, both suites run and results are recorded in the measurement spreadsheet with task ID, iteration, structural pass/fail per rule, API acceptance pass/fail, and retry count.*

[EVO-003] [Must]

Each transition between iterations shall be documented with: (a) which components changed, (b) which remained identical, (c) which API acceptance tests failed during transition and why, (d) which structural tests failed and why, (e) proportion of transition performed by AI agent versus manual, (f) what the AI agent produced incorrectly and how it was corrected, (g) retry counts per task during the transition.

Rationale: *Migration documentation captures the quantifiable evidence this project exists to produce.*

Verification: *Document exists for each transition with all seven elements.*

6. Non-Functional Requirements

ID	Pri	Requirement	Verification
NFR-001	**Must**	The system shall produce an updated composite score within 120 seconds under normal conditions.	Measure P95 latency. Verify < 120s.
NFR-002	**Must**	All Must-priority requirements shall have automated API acceptance tests. Combined coverage ≥ 80% on domain and application logic.	Coverage analysis on specified layers.

NFR-003	**Must**	All configurable parameters referenced in requirements shall be changeable without code modification.	Change parameter. Verify effect without rebuild.
NFR-004	**Should**	Data at rest containing assessments, decisions, or positions shall be encrypted.	Verify storage encryption.
NFR-005	**Should**	The system shall recover to operational state within 15 minutes of single-component failure.	Kill component. Measure recovery.

7. Iteration Plan

Six iterations. Each implements Must requirements plus specific Should requirements introduced as controlled change tasks. The API acceptance test suite (EVO-001a) is the constant. The architecture pattern is the variable. CQRS (Command Query Responsibility Segregation) is a tactical pattern prescribed from Iteration 3 onward: command handlers shall not return domain state, query handlers shall not modify state. This separation is structurally enforced in Iteration 4 where Event Sourcing produces distinct write (event stream) and read (projection) models. DDD tactical patterns (aggregates, value objects, domain events, domain services) are applied from Iteration 3 onward within the domain layer. Event Sourcing is a persistence strategy introduced in Iteration 4. Pre-registered prediction per iteration: each iteration specifies its expected position in the functional correctness × architectural compliance matrix before data collection begins. Predictions that fail are reported as findings.

7.1 Iteration 1 — Transaction Script (Baseline)

Scope: All Must requirements. No deliberate architectural structure beyond framework conventions (ASP.NET controllers, dependency injection container, project/solution organization). Procedural services, flat models, direct data access.

Deliverable: Working system. Green API acceptance tests. Trivial structural tests (no circular references). Baseline measurement data.

Prediction: Structural pass trivially high (no meaningful rules to violate). API acceptance pass rate establishes baseline. Expected quadrant: structural pass + API acceptance pass for most tasks.

Structural tests

No circular project references. No other architectural rules to enforce.

Data collected per AI task

Structural: pass/fail. API acceptance: pass/fail. Retry count per task.

7.2 Iteration 2 — Vertical Slice Architecture (Refactor)

Relationship: Refactor from Iteration 1. Git history continuous.

Scope: Same Must requirements. Code reorganised into feature slices.

Prediction: Moderate structural pass rate (slice isolation is a soft boundary). API acceptance pass rate comparable to baseline. CT-03 (cross-cutting) expected to produce per-slice duplication rather than shared abstraction.

Structural tests

Each feature slice is self-contained: no feature references another feature's internal types. Slice boundary definitions (namespace or project conventions) are specified in the iteration's agent.md.

Change tasks applied after refactor

1. **CT-01:** Add a new signal source type (new capability within Signal Ingestion).
2. **CT-02:** Modify composite scoring to support an additional weighting scheme (behaviour change within Classification).
3. **CT-03:** Add correlation ID logging across all operations (cross-cutting, all functional areas).

Data collected

Per AI task: structural pass/fail, API acceptance pass/fail, retry count. Per CT: files modified, API acceptance tests broken before correction. For CT-03 (cross-cutting): record whether the AI implemented the cross-cutting concern via shared abstraction or via per-slice duplication. EVO-003 transition document.

7.3 Iteration 3 — Clean Architecture with Rich Domain Model (Rewrite)

Relationship: Rewrite. API acceptance test suite carried forward unchanged.

Scope: All Must requirements reimplemented with strict layering: domain (zero dependencies, rich model), application (use case orchestration), infrastructure (adapters), presentation (API).

Prediction: Lower structural first-pass rate than Iterations 1–2 (strict layering rules are harder to satisfy on first attempt). API acceptance pass rate comparable or higher if constraints guide correctness. CQRS command/query separation expected AI-fragile.

Structural tests

Domain shall not reference Application, Infrastructure, or Presentation. Application shall not reference Infrastructure or Presentation. No concrete infrastructure type in Domain or Application. Command handlers shall not return domain state.

Should requirements introduced as change tasks

1. **CT-04:** SIG-005 — backpressure handling (cross-cutting resilience).
2. **CT-05:** DEC-004 — human-approval workflow (cross-boundary: Decision → Position Construction).
3. **CT-06:** RSK-002 — false positive budget (new domain concept, cross-context impact).
4. **CT-07:** OBS-002 — AI reasoning traces (observability addition to Classification).
5. **CT-08:** Add a new signal source adapter implementing the existing ingestion port interface. Verify no modification to the port interface or domain layer (extensibility proof via adapter pattern).

Data collected

Per AI task: structural pass/fail per rule, API acceptance pass/fail, retry count. Per CT: files modified, tests broken. EVO-003 transition document.

7.4 Iteration 4 — Clean Architecture + Event Sourcing (Refactor)

Relationship: Refactor of Iteration 3. Domain and application layers must not change. Only persistence layer in infrastructure changes.

Scope: Replace state-based persistence with event-sourced persistence. API acceptance suite passes without modification.

Prediction: Structural first-pass rate lower than Iteration 3 (Event Sourcing adds unfamiliar persistence patterns). Infrastructure-only change should yield zero files modified outside Infrastructure (dependency inversion holds). AI expected to attempt direct event stream queries at least once.

Structural tests

All Iteration 3 structural rules carry forward. Additionally: no application handler shall query the event stream directly (all reads through projections). Event store is append-only at the infrastructure boundary.

Critical measurement

Files changed outside the Infrastructure layer. This is the dependency inversion proof (Insight 5).

Should requirements introduced

1. **CT-09:** ANA-002 — return attribution (leverages event stream).
2. **CT-10:** AUD-002 — tamper-evident audit (natural on event stream).
3. **CT-11:** Add configurable position sizing multiplier based on signal conviction (domain enrichment in Position Construction). This domain behavior change is orthogonal to persistence. Files changed outside Domain and Application layers constitute a dependency inversion violation.

Data collected

Git diff: files changed outside Infrastructure (Insight 5). Per AI task: structural + API acceptance pass/fail, retry count. Whether AI understood “domain must not change” constraint (Insight 3). Whether the AI implemented CQRS as structurally required by Event Sourcing, or attempted to query the event stream directly. EVO-003 document with annotated list of any abstraction leaks.

7.5 Iteration 5 — Modular Monolith (Clean Architecture Concepts + Vertical Slice)

Relationship: Restructure of Iteration 3 into independently bounded modules with enforced isolation and inter-module contracts. Modules are internally organized as vertical slices following Clean Architecture dependency rules at module boundaries.

Scope: Seven functional areas (Section 4) mapped to independently bounded modules. Module boundaries, contract assemblies, and the mapping from functional areas to modules are defined in the iteration’s CLAUDE.md. All Must requirements reimplemented within the modular structure. API acceptance suite passes without modification.

Prediction: Structural pass rate comparable to Iteration 3 (module boundaries are enforceable via project references). Cross-module leak rate lower than Iteration 3 cross-layer leaks due to coarser, more visible boundaries. API acceptance pass rate stable.

Structural tests

All Iteration 3 structural rules carry forward within each module. Additionally: no direct code reference between modules. All inter-module communication through published module contracts. Each module's internal vertical slices are self-contained. Module boundary definitions are specified in the iteration's CLAUDE.md.

Should requirements introduced

1. **CT-15:** SEC-003 — threat model (security documentation).
2. **CT-13:** POS-004 — generate multi-leg options strategies (domain enrichment).
3. **CT-12:** EXT-003 — add new exit trigger type (extensibility proof).

Data collected

Cross-module boundary violations by AI (Insight 4). Sequential degradation: structural pass rate after each of CT-15, CT-13, CT-12 in sequence (Insight 2). EVO-003 document.

7.6 Iteration 6 — Service Extraction (Decision Engine)

Relationship: Transform Decision Engine in a standalone service from Iteration 5, communicating with the remaining monolith via a defined contract (HTTP or in-memory message bus, configurable). All other modules remain in-process.

What this iteration demonstrates

1. Service boundary enforcement: the extracted Decision Engine operates as an independent deployable with its own test suite.
2. Contract-first communication: all interaction between the extracted service and the monolith passes through a versioned contract.
3. Distributed change propagation: modifications to the service contract must be reflected on both sides.
4. Operational readiness: structured logging, distributed tracing, health checks across service boundaries.

Prediction: Structural pass rate comparable to Iteration 5 for in-process rules. Cross-service contract propagation (CT-17) expected to produce structural failures. AI expected to struggle with bidirectional contract updates.

Structural tests

No module references another module's internals. All Iteration 5 structural rules carry forward. Additionally: All communication between the service and the monolith through the serialized service contract (HTTP or message bus). Decision Engine's internal types are not transitively reachable from the monolith's dependency graph.

Should requirements introduced

1. **CT-14:** OBS-003 — alert across modules.
2. **CT-16:** RSK-003 — liquidity reserve (domain enrichment).
3. **CT-17:** Modify the Decision Engine service contract (add a field to the classification response). Verify the AI updates both the service and the consuming monolith.

Data collected

Cross-service contract violations by AI (Insight 4). Sequential degradation: structural pass rate after each of CT-14, CT-16, CT-17 in sequence (Insight 2). Whether the AI correctly propagated contract changes to both sides (CT-17). EVO-003 document.

8. Harness Instrumentation

Six measurement instruments addressing a single question: given that AI coding agents increasingly generate, modify, and evolve production code, which architectural constraints do they reliably respect, which do they systematically violate, and what is the economic cost of enforcement? Each measurement is objective. The AI agent's operational context (Section 8.6) is a controlled variable. The retry protocol (Section 8.7) defines when a result is recorded.

8.1 Insight 1: Taxonomy of AI-Compatible vs AI-Hostile Architectural Rules

Question: Which architectural constraints do AI agents reliably respect, and which do they consistently violate?

Measurement method

1. Each iteration defines specific structural rules (Section 7, structural tests per iteration).
2. After each AI task (following the retry protocol in 8.7), run the structural suite. Record pass/fail per rule.
3. After all iterations, aggregate pass rate per rule type across all tasks where that rule was active.
4. Classify: AI-reliable (>90% final pass rate), AI-fragile (50–90%), AI-hostile (<50%). The taxonomy shall additionally be computed at alternative thresholds (85/45 and 95/55). If classifications shift between threshold sets, this instability shall be reported.

Additionally, record the API acceptance suite result per task

This produces a 2x2 matrix per task: structural pass/fail × API acceptance pass/fail. Aggregate across all tasks to answer: does structural compliance correlate with functional correctness? If AI-generated code that passes structural tests also passes API acceptance tests at a higher rate, then architectural enforcement actively improves functional correctness under AI development.

Recording format

Spreadsheet: Iteration | Task ID | Task Description | Rule ID | Rule Type | Structural Pass/Fail | API Acceptance Pass/Fail | Retry Count.

8.2 Insight 2: Architectural Integrity Degradation Under Sequential AI Changes

Question: Does architectural integrity degrade as AI agents make sequential changes to the same codebase?

Measurement method

1. Within each iteration, change tasks are applied sequentially.
2. After each task (post-retry), record the structural suite pass rate (% of rules passing) AND the API acceptance suite pass rate.

3. Plot both rates (y-axis) against sequential task number (x-axis) per iteration.
4. Declining structural curve = architecture erosion. Measurable degradation is defined as a decline of >10 percentage points in structural suite pass rate across sequential change tasks within a single iteration. This threshold is pre-registered before data collection. Declining API acceptance curve = functional regression. Correlation between the two curves indicates whether architecture erosion causes functional degradation.

Why this matters

If degradation is measurable, AI-maintained systems need periodic architectural realignment — a new operational practice. If not, AI agents are more structurally consistent than human teams. Either finding is novel.

Recording format

The raw task data (Attempt 0 and Final pass/fail results) shall be recorded sequentially in the master measurement spreadsheet as strictly defined in requirement [RTP-003]. The aggregated visual output shall be a line chart per iteration plotting the structural suite pass rate (%) and API acceptance suite pass rate (%) on the y-axis against the sequential task number on the x-axis.

8.3 Insight 3: Can AI Agents Perform Clean Architectural Migration?

Question: When instructed to change only the infrastructure layer, does the AI agent respect the boundary?

Measurement method

1. Iteration 3 → 4 transition is the test case.
2. Instruct the AI: “Replace persistence with event sourcing. Domain and application layers must not change.”
3. After migration, git diff --stat against Iteration 3. Count files changed outside Infrastructure.
4. Classify each leak: domain entity modified, application handler modified, shared contract changed.
5. Verify the API acceptance suite passes without modification. Any API acceptance failure indicates the migration changed behaviour, not just structure.

Recording format

Git diff output. Annotated list of non-infrastructure changes. API acceptance suite pass/fail.

8.4 Insight 4: Bounded Context Isolation Under AI-Driven Cross-Boundary Changes

Question: When a change must propagate across functional areas, does the AI agent respect the context boundaries?

Measurement method

1. Cross-boundary tasks: CT-05 (Decision → Position Construction), CT-06 (Risk → Decision), CT-14 (all modules), CT-17 (cross-service contract modification).
2. Per task: count direct references introduced between contexts that should communicate only through published contracts. Zero = respected. Positive = leak.

3. Classify leaks: concrete type imported, repository accessed directly, event bus bypassed with method call.
4. Compare leak counts between iterations. Does Modular Monolith (Iteration 5) produce fewer leaks than Clean Architecture (Iteration 3)? Does Service Extraction (Iteration 6) produce fewer leaks than Modular Monolith?
5. For each leak, record whether the API acceptance tests caught the violation. If API acceptance tests pass despite boundary leaks, the leak is architecturally concerning but functionally invisible — the most dangerous kind.

Recording format

Per cross-boundary task: list of cross-context references by type. API acceptance test result. Comparison table across iterations.

8.5 Insight 5: Event Sourcing Swap as Dependency Inversion Proof

Question: Is the Iteration 3 → 4 persistence swap contained to the infrastructure layer?

Measurement method

Specific instance of Insight 3 with binary outcome. Files changed outside Infrastructure. API acceptance suite pass/fail without modification. Reported separately for article weight.

Recording format

Single number: files changed outside Infrastructure. API acceptance suite result. If nonzero changes, annotated list of leaks classified by type.

8.6 Agent Context Strategy

The AI agent's operational environment shall be a controlled variable across all iterations. The following conditions shall be standardised and documented to ensure measurement validity.

[ACX-001] **[Must]**

The AI agent context for every task across all iterations shall be documented and held constant. The documentation shall specify: (a) whether the agent has full repository access or is restricted to specific file inputs; (b) whether the agent session is fresh (no context from prior iterations) or continuous; (c) whether the agent has Language Server Protocol (LSP) integration, compiler feedback, or test runner access; (d) the maximum context window or file set provided per task;

Rationale: *The AI agent's environment directly affects its output quality. If the agent has LSP in one iteration but not another, or sees the full solution in one task but only a single file in another, the measurements are not comparable. Standardising the context makes the architecture pattern the only variable under test.*

Verification: *A document exists specifying all four conditions. All measurement spreadsheet entries include a field confirming the conditions were met for that task. Any deviation is logged with justification. The CI/CD test runner must output an LSP trace log for every task. The pipeline shall automatically parse the log to verify the presence of JSON-RPC diagnostic exchanges. Any task run with an empty or inactive LSP log shall be automatically marked as invalid to prevent data corruption.*

[ACX-002] **[Must]**

A standardised prompt baseline shall be established before Iteration 1 begins. The baseline shall define: (a) the project context file (CLAUDE.md) structure and content per iteration, including: project identity, architecture pattern, structural rules, and build/test commands; and constraints (ex. “Existing API acceptance tests must continue to pass without modification."); (b) the task prompt structure used for all tasks;

The CLAUDE.md is updated per iteration. The task prompt structure and session protocol are constant across all iterations. For each iteration, the CLAUDE.md characteristics shall be recorded: (a) word count, (b) number of explicit architectural rules, (c) specificity level (low/medium/high). These measurements enable assessment of whether performance differences correlate with context quality rather than architectural pattern.

Rationale: *Claude Code loads CLAUDE.md at session start as persistent project context. Architectural rules belong in project context, not in task prompts, reflecting standard team workflows. Separating what changes (CLAUDE.md per iteration) from what stays constant (prompt structure, session protocol) isolates the architecture pattern as the experimental variable.*

Verification: *CLAUDE.md exists per iteration. Task prompts logged alongside measurement data conform to the defined structure. Session protocol adherence confirmed per task.*

8.7 AI Retry Protocol

When an AI agent produces code that fails structural or API acceptance tests, the agent may be given the opportunity to self-correct by reading the error output. This section defines the protocol to ensure consistent recording.

[RTP-001] **[Must]**

After each AI task, both the structural test suite and the API acceptance test suite shall be executed. The AI coding agent shall work autonomously, including its internal test-fix loops. Attempt 0 is defined as the state when the agent first declares its task complete. External test suites (structural + API acceptance) are then run. If either suite fails, the test output is provided to the agent as a new prompt. The agent works autonomously again until it declares done. This constitutes one human-initiated retry.

Rationale: *AI agents frequently self-correct when given compiler or test output. Measuring only the first attempt understates the agent’s effective capability. Measuring only the final attempt after unlimited retries overstates it. A bounded retry protocol captures realistic AI-assisted workflow.*

Verification: *The protocol is followed for every task. Retry count is recorded.*

[RTP-002] **[Must]**

The maximum number of human-initiated retry attempts per task shall be three (3). After Attempt 0 plus three human-initiated retries (four total agent runs), the result is final. The agent’s internal retry count (visible in session logs) shall be recorded as supplementary data per attempt.

Rationale: *Three human-initiated retries reflects a realistic workflow where a developer would intervene after 3–4 failed cycles. The agent’s autonomous internal retries within each attempt are not bounded, reflecting actual Claude Code behavior. This aligns with the pass@1 / pass@k evaluation framework used by SWE-bench and other AI coding benchmarks.*

Verification: *No task in the spreadsheet has a retry count exceeding 3.*

[RTP-003] **[Must]**

Each task shall be recorded in the measurement spreadsheet with the following fields: (a) Task ID, (b) Iteration, (c) Task Description, (d) Task Type (new capability / behaviour change / cross-cutting / cross-boundary / plugin / agent), (e) Attempt 0 structural result (pass/fail per rule), (f) Attempt 0 API acceptance result (pass/fail), (g) Final structural result after retries, (h) Final API acceptance result after retries, (i) Total retry count (0–3), (j) Manual corrections required after final attempt (yes/no, with description if yes), (k) Attempt 0 Input Tokens (l) Attempt 0 Output Tokens (m) Final Input Tokens (Cumulative) (n) Final Output Tokens (Cumulative) (o) Task Mutation Score % (p) New SAST Vulnerabilities (Critical/High count) (q) Delta in Maintainability Index (r) Task Relationship Type (new-implementation / refactor / rewrite / localized-refactor)

Rationale: *Recording both the initial attempt and the final result enables analysis of the AI's first-pass accuracy vs. self-correction capability. Tracking token consumption, mutation scores, and SAST vulnerabilities ensures the economic and operational costs of the AI's code are measured alongside its architectural compliance.*

Verification: *Spreadsheet contains all specified fields (a through r) for every task across all iterations.*

[RTP-004] **[Must]**

A task shall be classified in the Insight 1 taxonomy as follows: (a) AI-reliable: final structural pass rate >90% across all tasks where that rule was active; (b) AI-fragile: final pass rate 50–90%; (c) AI-hostile: final pass rate <50%. Separately, report the Attempt 0 (first-pass) rates for the same buckets to show the self-correction effect.

Rationale: *The dual reporting (first-pass vs post-retry) is essential. A rule with 40% first-pass rate but 85% post-retry rate is AI-fragile on first contact but AI-reliable with feedback. A rule with 40% first-pass and 45% post-retry is genuinely AI-hostile — feedback does not help. These are qualitatively different findings.*

Verification: *Insight 1 table includes both first-pass and post-retry columns per rule type.*

8.8 Insight 6: Operational & Economic Viability Under AI Development

Question: Do strict architectural patterns increase the operational cost (token consumption), latent security debt, and human cognitive load of AI-generated code, or do they constrain the AI into producing more economical, secure, and maintainable solutions?

Measurement method

1. **Token Economics:** For every AI attempt, record the total context tokens (input) and generated tokens (output). Calculate the aggregate "Token-to-Pass Ratio" for each architectural iteration to determine the true cost of AI-driven maintenance.
2. **True Test Quality (Mutation Score):** While NFR-002 mandates ≥80% raw coverage, AI agents frequently generate tautological tests to satisfy coverage thresholds. Run a mutation testing framework (e.g., Stryker) on the domain and application logic modified during the task. Mutation testing shall be executed on final attempt outputs per task, not on every retry attempt, to manage computational cost while preserving cross-iteration comparison validity. Record the Mutation Score % against the raw coverage %.
3. **Latent Security Debt (SAST):** Execute a Static Application Security Testing (SAST) scan on Attempt 0 and the Final Attempt. Record the delta of newly introduced Critical or

High vulnerabilities per task (e.g., implicit SQL injection, insecure deserialization) to see if specific architectural boundaries organically quarantine insecure AI hallucinations.

4. **Maintainability Delta and Regeneration Cost:** Calculate the Cyclomatic Complexity and Maintainability Index of the specific files modified by the AI agent, comparing the pre-task and post-task states. For any module where structural integrity degrades below a configurable threshold after sequential changes, record the token cost and time to regenerate the module from spec versus the token cost and time to repair it. This provides evidence for or against the disposable software thesis.

Why this matters

In the current industry phase, context window consumption and inference compute are massive enterprise cost drivers. Furthermore, the handover between AI generation and human maintenance is the primary friction point in modern SDLCs. If a specific architecture (e.g., Clean Architecture) forces the AI to consume 4x the tokens and produce highly complex, hard-to-read code compared to a Modular Monolith, the theoretical architectural purity is outweighed by the economic and operational drag. Additionally, identifying which architecture best prevents AI from introducing silent security vulnerabilities is an urgent industry need.

Recording format

The telemetry for operational and economic viability shall be recorded in the master measurement spreadsheet, specifically populating fields (k) through (q) as strictly defined in requirement [RTP-003].

[OEV-001] **[Must]**

The CI/CD pipeline executing the structural and API acceptance test suites shall automatically collect the metrics for mutation score, SAST vulnerabilities, and maintainability index without requiring manual human calculation.

Rationale: *Manual calculation of these metrics across hundreds of AI task attempts introduces unacceptable overhead and risks data inconsistency. Automation ensures the telemetry is passively and accurately collected.*

Verification: *Demonstrate the CI pipeline successfully populating these specific metrics into a log or data store after a single test task.*

8.9 Insight 7: Spec Precision as AI Pass Rates

Question: Does the precision of the requirement specification correlate with AI implementation quality?

Measurement method

For each functional requirement in Sections 5–6, classify the specification precision: (a) quantitative formula (e.g., CLS-002, CLS-006), (b) qualitative description (e.g., CLS-003), (c) structural constraint (e.g., POS-001). Compare AI first-pass and final pass rates per precision level across all iterations. If precisely specified requirements produce consistently higher pass rates, that constitutes evidence for spec-driven development effectiveness.

The variation in specification precision across requirements is deliberate and constitutes the independent variable for this insight. Requirements are not normalized to uniform precision.

8.10 Experimental Variable Registry

The following registry documents every variable known to influence AI coding agent output quality, its chosen value in this experiment, and its treatment (controlled, measured, or stated limitation). Variables not controlled or measured are acknowledged as threats to external validity.

Variable	Chosen Value	Alternatives	Treatment
F1. AI Model			
Model	Claude (via Claude Code), version pinned at experiment start	GPT-4.x/5, Gemini, open-source (Llama, Codestral)	Stated limitation. Experiment design is model-agnostic. Explicit invitation for replication.
Model Version	Fixed at experiment start	Newer versions released mid-experiment	Controlled. Version documented in ACX-001. Version change mid-experiment invalidates affected iteration.
Temperature / Sampling	Default Claude Code settings	Lower (more deterministic), higher (more creative)	Controlled. Default settings. Documented in ACX-001.
F2. Agent Orchestration			
Agent Mode	Single-agent (Claude Code CLI)	Multi-agent (coordinator + workers), sub-agents per module	Stated limitation. Multi-agent could alter Insights 4 and 6. Phase 2 future work.
Agent Permissions	Standard (with confirmation)	Auto-approve (wildcard), Plan-only (read-only)	Controlled. Consistent across iterations. Documented in ACX-001.
Session Continuity	Fresh session per iteration	Continuous session, persistent memory (MEMORY.md)	Controlled (ACX-001). No cross-iteration memory.
Subagent Usage	No subagents	Claude Code subagents for parallel execution	Stated limitation. Single-operator constraint. Phase 2.
F3. Tooling & Infrastructure			

LSP Integration	Enabled (.NET via OmniSharp/csharp-ls)	Disabled (text-only, grep-based navigation)	Measured. ACX-003 subset with LSP disabled isolates tooling vs pattern effect.
Compiler Feedback	Enabled (dotnet build errors fed back)	Disabled (no build feedback)	Controlled. Consistent across iterations. Documented in ACX-001.
Test Runner Access	Enabled (agent can run dotnet test)	Disabled (human runs tests)	Controlled. Consistent across iterations.
IDE vs CLI	CLI (Claude Code terminal)	IDE (Cursor, Windsurf, VS Code + Copilot)	Stated limitation. CLI chosen for reproducibility and scriptability.
MCP Servers	None (built-in tools only)	External MCP servers (Git, docs, architecture)	Controlled. No external MCP servers. Built-in tools only.
F4. Context Engineering			
CLAUDE.md Quality	Measured per iteration (word count, rule count, specificity)	Minimal (build commands only), Maximal (full spec), None	Measured (D2). Enables assessment of context vs architecture effect.
Task Prompt Structure	Standardised (ACX-002)	Varied (detailed for complex, brief for simple)	Controlled. Standardised per ACX-002.
Context Window	Natural (whatever Claude Code loads)	Truncated, Force-fed full codebase	Documented in ACX-001. Natural behaviour for ecological validity.
Spec Precision	Mixed (formulas, qualitative, structural)	Uniformly precise, uniformly vague	Measured (Insight 7). Pass rates compared per precision level.
F5. Language, Framework & Domain			
Language	C# / .NET 8+	Java/Spring, TypeScript/Node, Python, Rust, Go	Stated limitation. Operator expertise: 10+ years .NET. Framework reusable for any language.
Framework	ASP.NET (Web API)	Minimal API, gRPC, serverless	Controlled. Consistent across iterations.

Domain Complexity	Medium-high (trading system)	Low (CRUD), Very high (distributed tx)	Acknowledged. Real system, not synthetic benchmark.
Codebase Size	Small-to-medium (single-service scope)	Large (100k+ LOC), Micro (<1k LOC)	Stated limitation. Scale effects may not manifest.
F6. Architectural Pattern Variables			
Pattern Selection	6 traditional patterns (TS, VSA, Clean, Clean+ES, Modular Monolith, Service Extraction)	AI-native (modular regeneration, spec-anchored governance, self-modifying)	Stated limitation. Traditional = Phase 1 foundation. AI-native = Phase 2.
DDD Tactical Patterns	Applied within domain layer, Iterations 3+	No DDD (anemic throughout), Full DDD everywhere	Partially confounded with architecture variable. Acknowledged.
Structural Test Strictness	Varies by iteration (none for Iter 1, strict for Iter 3+)	Uniform strictness across all iterations	Measured. ACX-003 tests whether compliance comes from rules or code inference.
F7. Human Factors			
Researcher Expertise	Deep (.NET, DDD, Clean Architecture, 10+ years)	Novice, Generalist	Stated limitation. Context quality correlates with expertise.
Prompt Writing Skill	Expert (refined through extensive Claude Code usage)	Novice (first-time agent user), Standardised templates	Partially controlled (ACX-002 standardises structure). Expertise acknowledged.
Evaluation Bias	Researcher evaluates own experiment	Independent evaluator, Automated-only evaluation	Mitigated: NetArchTest automation. Raw data published for reclassification.

9. Validation Event Set

Ten historical events. Includes true positives, false positives, and one correctly missed event.

Event	Date	Type	Expected
Volmageddon	Feb 5, 2018	Liquidity	Deploy by Feb 2. True positive.

Soleimani Strike	Jan 3, 2020	Geopolitical	Signal fires. False positive.
COVID-19	Feb 21, 2020	Structural Break	Deploy by Feb 20. True positive.
Russia/Ukraine	Feb 24, 2022	Geopolitical	Deploy by Feb 22. True positive.
SVB Collapse	Mar 10, 2023	Structural Break	Deploy by Mar 9. True positive.
Yen Carry Unwind	Aug 5, 2024	Liquidity	Deploy by Aug 1. True positive.
Iran→Israel	Oct 1, 2024	Geopolitical	False positive.
FOMC Surprise	Dec 18, 2024	Macro Shock	Below threshold. Correctly missed.
Liberation Day	Apr 2, 2025	Macro Shock	Deploy by Apr 1. True positive.
Iran Strikes	Feb 28, 2026	Geopolitical	Deploy by Feb 27. True positive.

10. Change Task Register

Controlled requirement implementations. Each has a type, governing which measurement dimensions it produces.

ID	Description	Type	Req	Iter	Insights
CT-01	Add new signal source type	New capability	SIG-001	2	1, 2
CT-02	Add additional weighting scheme	Behaviour change	CLS-002	2	1, 2
CT-03	Add correlation ID logging	Cross-cutting	OBS-001	2	1, 4
CT-04	Implement backpressure	Cross-cutting resilience	SIG-005	3	1, 2
CT-05	Human-approval workflow	Cross-boundary	DEC-004	3	1, 4
CT-06	False positive budget	Cross-context domain	RSK-002	3	1, 4
CT-07	AI reasoning traces	Observability	OBS-002	3	1, 2
CT-08	New signal source adapter (extensibility via adapter pattern)	Adapter extensibility	SIG-001	3	1, 4
CT-09	Return attribution	Analytics	ANA-002	4	1
CT-10	Tamper-evident audit	Security	AUD-002	4	1

CT-11	Domain behavior change orthogonal to persistence	Domain enrichment	POS-002	4	1, 2
CT-12	New exit trigger type	Extensibility	EXT-003	5	1
CT-13	Multi-leg options strategies	Domain enrichment	POS-004	5	1
CT-14	Alerting across modules	Cross-module	OBS-003	6	1, 4
CT-15	Threat model	Documentation	SEC-003	5	—
CT-16	Liquidity reserve	Domain enrichment	RSK-003	6	1
CT-17	Service contract modification	Cross-service	DEC-001	6	1, 4

11. System-Level Acceptance Criteria

1. The API acceptance test suite (EVO-001a) passes on all completed iterations without test code modification.
2. Structural tests (EVO-002) exist per iteration and record at least one AI-generated violation during development.
3. Event replay of Iran Feb 2026 produces deploy signal on Feb 27 ± 1 day in all iterations.
4. All five prompt injection tests (CLS-005) rejected in all iterations.
5. Iteration 3→4 migration documented with exact file count outside Infrastructure.
6. Migration documentation (EVO-003) exists for each transition with all seven elements.
7. Agent context strategy (ACX-001, ACX-002) documented before Iteration 1.
8. Measurement spreadsheet contains all RTP-003 fields for every task across all iterations.
9. Insight 1 taxonomy table includes both first-pass and post-retry columns.
10. At least 5 dual-condition prompt experiments (ACX-003) recorded per iteration where attempted.
11. The automated CI/CD pipeline successfully captures and records the Operational & Economic Viability telemetry (mutation score, SAST findings, maintainability index) without manual intervention (OEV-001).
12. For CLS-002 and CLS-006, API acceptance tests shall include hand-calculated expected outputs verified against the formulas defined in this specification.

END OF SRS v2.3.0